Implementación, automatización y gestión de una infraestructura de microservicios en AWS EKS: Un enfoque basado en Terraform y Portainer



TFG 2º de ASIR IES La Marisma

Jesús Carlos Mora Mesa

Índice

Capítulo 1: Marco Teórico....4
1.1. La contenerización como siguiente paso de la virtualización....4
1.2. La necesidad de la orquestación con Kubernetes....5
1.3. Portainer: La interfaz de gestión de Kubernetes....7
1.4. Relevancia empresarial: Seguridad, administración y eficiencia....8
Capítulo 2: Arquitectura del proyecto y contenerización local....9
2.1. Adopción de la Arquitectura Monorepo y Separación de Contextos....10
2.2. Diseño del Entorno de Desarrollo completo con Docker....11
2.2.1. Optimización de Imágenes con Multi-stage Builds....11
2.2.2. Orquestación Modular mediante múltiples Docker Compose....12
2.2.3. Developer Experience (DX) y Scripts de Automatización....12
2.3. Gestión de Configuración y Seguridad: El paradigma de las Variables de Entorno....14
2.3.1. Aislamiento e Inyección en Contenedores....14
2.3.2. Seguridad y Control de Versiones....15
2.3.3. Preparación para la Orquestación....16
2.4. Homogeneización de Entornos y Gestión de Binarios Nativos....16
2.5. Resultado y arquitectura en funcionamiento....17
Capítulo 3: Integración Continua (CI) y Gestión de Artefactos....19
3.1. Transición del desarrollo local a la automatización (Pipelines)....19
3.2. Seguridad en la Nube: De archivos .env a GitHub Secrets....19
3.3. Estrategia de Ramas, Control de Calidad y Prevención de Deuda Técnica....20
3.4. Optimización, Seguridad (DevSecOps) y Publicación....20
3.5. Justificación Técnica y Aportes de Valor del Pipeline....24
3.6. Resolución de Conflictos Multiplataforma (Troubleshooting CI)....25
Capítulo 4: Infraestructura como Código (IaC) y Despliegue en AWS EKS....26
4.1. Preparación del Entorno de Aprovisionamiento....26
4.2. Aprovisionamiento detallado de la infraestructura con Terraform....27
4.2.1. Bloque de configuración de Terraform (terraform {})....27
4.2.2. Bloque del Proveedor de AWS (provider "aws" {})....28
4.2.3. Proceso de Inicialización (terraform init)....29
4.3. Arquitectura de Red: Despliegue de la VPC....29
4.3.1. Segmentación de Red y Alta Disponibilidad....29
4.3.2. Gestión de Salida a Internet y NAT Gateway....30
4.3.3. Etiquetas de Descubrimiento de Kubernetes....30
4.4. Orquestación y Cómputo: Despliegue del Clúster Amazon EKS....31
4.4.1. El Plano de Control y la Gestión del Ciclo de Vida....31
4.4.2. Integración Segura con la Topología de Red....32
4.4.3. Cómputo: Managed Node Groups y Alta Disponibilidad....32
4.5. Orquestación de Contenedores: Despliegue Lógico de la Aplicación....33
Capítulo 5: Gestión Visual del Clúster con Portainer....38
5.1. Introducción a Portainer....38
5.2. Proceso de Instalación....38
5.2.1. Configuración del Repositorio....38
5.2.2. Despliegue en el clúster....38
5.2.3. Acceso y Seguridad Inicial....39
5.3. Monitorización y Administración del Proyecto....39
5.3.1. Dashboard General....39
5.3.2. Gestión de Aplicaciones (Namespaces)....40
5.3.3. Inspección de Logs y Consola....40
5.3.4. Networking e Ingress....41

Capítulo 6: Conclusiones y líneas de mejora futuras.....42
6.1. Conclusiones técnicas.....42
6.2. Líneas de mejora futuras.....42
6.4. Diagrama de arquitectura final.....43

Capítulo 1: Marco Teórico

1.1. La contenerización como siguiente paso de la virtualización.

La infraestructura de tecnologías de la información ha experimentado una transformación radical en la última década, impulsada por la necesidad de mayor agilidad, eficiencia y escalabilidad en el despliegue de aplicaciones. Tradicionalmente, el modelo imperante se basaba en la virtualización de hardware, donde un hipervisor permitía ejecutar múltiples sistemas operativos completos sobre un mismo servidor físico. Pongamos de ejemplo el servidor del instituto: Proxmox corre sobre Debian, pero podemos instalar cualquier sistema operativo que queramos, desde Windows 10 a sistemas operativos propios de contenedores LXC.

Si bien este modelo, representado por las máquinas virtuales, supuso un avance gigantesco respecto a los servidores "bare-metal" o dedicados, conlleva una sobrecarga inherente significativa. Cada máquina virtual requiere su propio sistema operativo completo, incluyendo kernel, bibliotecas y binarios, lo que resulta en un consumo elevado de recursos (CPU, RAM y almacenamiento) y tiempos de arranque que pueden oscilar entre segundos y minutos.

En contraposición a este modelo surge la contenerización, que representa el siguiente paso evolutivo en la abstracción de recursos. Para comprender la diferencia fundamental, debemos observar qué nivel del sistema se está duplicando:

En la virtualización tradicional, esta tecnología 'abstrae el hardware'; es decir, el software divide el servidor físico en varias porciones, y en cada una de ellas debemos instalar un sistema operativo completo y pesado para que funcione. Es como si construyéramos varias casas independientes, cada una con sus propios cimientos y tejado, lo cual consume muchos recursos.

Por el contrario, la contenerización da un paso más allá y 'abstrae el sistema operativo'. En este modelo, no duplicamos el sistema operativo completo, sino que todos los contenedores comparten el mismo núcleo, el kernel, del servidor anfitrión, manteniéndose aislados unos de otros solo en el nivel superior donde se ejecutan las aplicaciones. Siguiendo la analogía, esto se asemeja a un edificio de apartamentos: todos comparten los mismos cimientos y estructura (que sería el sistema operativo), pero cada inquilino tiene su propio espacio privado y seguro. Esto elimina la enorme

carga de mantener múltiples sistemas operativos encendidos, permitiendo que el sistema sea mucho más ligero y rápido.

En esencia, un contenedor es una unidad estándar de software que empaqueta el código y todas sus dependencias para que la aplicación se ejecute de manera rápida y fiable de un entorno informático a otro. Desde una perspectiva técnica, los contenedores comparten el kernel del sistema operativo anfitrión, pero se ejecutan en procesos aislados del espacio de usuario.

Esta capacidad de aislamiento se logra mediante características nativas del kernel de Linux, principalmente los *cgroups* (control groups), que limitan y monitorizan el uso de recursos, es decir, ponen límites y contadores y se encargan de que un proceso no acapare todo el servidor; y los *namespaces*, que aíslan lo que el proceso puede ver del sistema (como la red, el sistema de archivos o los identificadores de proceso) engañando al proceso para que crea que es el único en el sistema o que tiene recursos exclusivos. Crean una "realidad paralela" para dicho proceso, haciendo que el contenedor no pueda ver ni matar procesos fuera de ella.

Al eliminar la necesidad de un sistema operativo invitado redundante para cada aplicación, los contenedores son órdenes de magnitud más ligeros que las máquinas virtuales, arrancan en milisegundos y permiten una densidad de aplicaciones mucho mayor en el mismo hardware físico.

Esta eficiencia es la piedra angular sobre la que se construyen las arquitecturas de microservicios modernas.

1.2. La necesidad de la orquestación con Kubernetes.

Si bien el contenedor individual resuelve el problema de la portabilidad y la eficiencia, introduce un nuevo desafío operativo cuando se traslada a un entorno de producción empresarial. Una aplicación moderna rara vez reside en un único contenedor; por el contrario, suele estar compuesta por decenas o cientos de contenedores interactuando entre sí (bases de datos, frontends, APIs, sistemas de caché). Gestionar estos contenedores manualmente, monitorizar su estado, asegurar su comunicación y garantizar que se reinicien en caso de fallo es una tarea humanamente inabarcable si la escala es demasiado grande, se necesitaba otra manera. Aquí es donde entra en juego el concepto de orquestación de contenedores.

El estándar en la industria para esta tarea es Kubernetes (a menudo abreviado como K8s). Kubernetes es una plataforma de código abierto para automatizar el despliegue, el escalado y la gestión de aplicaciones en contenedores. Funciona agrupando los contenedores que conforman una

aplicación en unidades lógicas para facilitar su administración y descubrimiento. Kubernetes actúa como el "director de orquesta", decidiendo en qué servidor físico o virtual debe ejecutarse cada contenedor basándose en los recursos disponibles, sustituyendo contenedores que fallan (self-healing), y distribuyendo el tráfico de red entre ellos (load balancing).

Sin embargo, la potencia de Kubernetes viene acompañada de una complejidad notoria. Su curva de aprendizaje es pronunciada, requiriendo que los administradores dominen una vasta cantidad de conceptos abstractos como los Pods, Deployments, Ingress, Services y ConfigMaps, además de interactuar casi exclusivamente a través de interfaces de línea de comandos y archivos de configuración en formato YAML extensos y propensos a errores sintácticos.

Para darle contexto al propósito de este proyecto, es imprescindible definir el paradigma bajo el cual operan herramientas como Kubernetes y Portainer: el concepto de Cloud Native. A menudo se confunde este término con simplemente alojar aplicaciones en servidores remotos, pero su significado es arquitectónico, no geográfico. Una aplicación es 'Nativa de la Nube' cuando ha sido diseñada desde su concepción para aprovechar las ventajas de elasticidad y escalabilidad que ofrecen los entornos modernos, independientemente de si se ejecuta en una nube pública como AWS o en un servidor local privado.

El enfoque Cloud Native propone abandonar las arquitecturas 'monolíticas' tradicionales, donde toda la aplicación era un único bloque de código gigante y difícil de modificar. En su lugar, aboga por descomponer los sistemas en piezas pequeñas, independientes y conectadas entre sí, llamadas microservicios, que se empaquetan dentro de contenedores. Esto permite que la aplicación sea modular: si una parte falla, el resto sigue funcionando; si una parte necesita más potencia, se puede escalar solo ese módulo sin duplicar toda la aplicación.

En esencia, adoptar una filosofía Cloud Native significa construir sistemas que son resistentes y automatizables. Mientras que en el modelo tradicional los administradores trataban a los servidores como piezas únicas e insustituibles que debían cuidarse manualmente, en el modelo Cloud Native la infraestructura es efímera y desechable. Gracias a la orquestación de contenedores, el sistema es capaz de sanarse a sí mismo, reiniciando servicios o moviéndolos de un servidor a otro automáticamente ante cualquier fallo, garantizando así una disponibilidad continua del servicio.

Para una empresa que busca adoptar tecnologías *Cloud Native*, esta complejidad técnica representa una barrera de entrada significativa, elevando los costes de formación y aumentando el riesgo de errores humanos en la configuración de la infraestructura crítica.

1.3. Portainer: La interfaz de gestión de Kubernetes.

En este contexto de alta complejidad técnica, Portainer surge como una solución fundamental para cerrar la brecha entre la potencia de la orquestación de contenedores y la operatividad humana. Portainer es una plataforma de gestión de contenedores universal y ligera que funciona, a su vez, dentro de un contenedor. Su propósito principal es simplificar drásticamente el despliegue y la administración de entornos Docker, Docker Swarm y, en el caso de este proyecto, Kubernetes. No se trata simplemente de una interfaz gráfica que le simplifique el trabajo al usuario, sino de una capa de abstracción operativa que interactúa directamente con la API del orquestador subyacente para traducir intenciones administrativas en acciones técnicas complejas.

La importancia de Portainer radica en su capacidad para "democratizar" el uso de Kubernetes. Traduce la complejidad de los archivos manifiestos YAML y los comandos de terminal kubectl en flujos de trabajo visuales e intuitivos. Esto permite que administradores de sistemas que no son expertos en la sintaxis profunda de Kubernetes puedan desplegar aplicaciones, gestionar volúmenes persistentes y configurar redes con confianza y seguridad. Por ejemplo, donde un despliegue tradicional en Kubernetes requeriría redactar manualmente definiciones para el Deployment, el Service y el Ingress, Portainer permite realizar estas acciones a través de un formulario unificado que valida los datos en tiempo real, reduciendo la posibilidad de errores de configuración que podrían dejar un servicio inoperativo.

Portainer actúa como un panel de control centralizado. En un entorno empresarial real, es común tener infraestructuras híbridas: algunos clústeres en servidores locales (on premise), otros en la nube pública y otros en el borde entre ambos (edge computing). Portainer permite conectar y visualizar todos estos entornos desde una única instancia, eliminando la necesidad de cambiar contextos o credenciales constantemente. Esta centralización es vital para la administración de estos entornos informáticos, ya que proporciona una visión completa del estado de salud de toda la infraestructura de contenedores de la organización en tiempo real, pero también permite comprobar “pieza a pieza” que todo funcione correctamente.

1.4. Relevancia empresarial: Seguridad, administración y eficiencia.

Más allá de la simplificación técnica, la implementación de Portainer en un entorno corporativo responde a necesidades críticas de seguridad y gestión de equipos. Uno de los mayores riesgos en la gestión de clústeres de Kubernetes "crudos" es el control de acceso. Por defecto, gestionar los permisos de quién puede ver, editar o borrar recursos en un clúster es una tarea complicada que implica la creación manual de certificados y roles RBAC (control de acceso basado en roles). Portainer abstrae esta complejidad ofreciendo un sistema de gestión de usuarios y equipos robusto e integrado. Permite a los administradores principales definir capa a capa (granularidad, en resumen) qué usuarios tienen acceso a qué clústeres o incluso a qué partes específicas de una aplicación (Namespaces), integrándose a menudo con directorios corporativos como LDAP o Active Directory.

Además, Portainer facilita la adopción de buenas prácticas DevOps y la estandarización de despliegues. A través de su funcionalidad de "App Templates" (plantillas de aplicaciones), los arquitectos de sistemas pueden predefinir configuraciones seguras y optimizadas para las aplicaciones que la empresa utiliza recurrentemente (bases de datos, servidores web, herramientas de monitorización). De esta forma, un desarrollador o un operador junior puede desplegar una instancia de base de datos compleja con un solo clic, garantizando que cumple con las normativas de la empresa, en lugar de intentar configurarla desde cero con posibles vulnerabilidades.

Finalmente, la visibilidad que aporta Portainer es un recurso fundamental para la auditoría y el mantenimiento. Ofrece acceso directo a los registros (logs) de los contenedores y a las consolas de comandos internas de los mismos directamente desde el navegador web, sin necesidad de establecer conexiones SSH complejas y potencialmente inseguras a los nodos del clúster. Esto acelera drásticamente los tiempos de diagnóstico y resolución de incidencias. Este factor impacta directamente en el MTTR (Mean Time To Resolution), métrica que calcula el tiempo promedio necesario para restaurar el servicio, reduciéndolo drásticamente gracias a la visibilidad inmediata que ofrece la herramienta.

En resumen, Portainer transforma Kubernetes de una tecnología potente pero inaccesible, o al menos técnicamente compleja, en una plataforma gestionable, segura y productiva apta para el ritmo y las exigencias del entorno empresarial actual.

Capítulo 2: Arquitectura del proyecto y contenerización local

Para comprender las decisiones técnicas detalladas a continuación, es necesario contextualizar el entorno de trabajo del proyecto, entorno que se ha convertido en real durante mis prácticas duales. Se nos pidió desarrollar una aplicación web con un equipo multidisciplinar compuesto por seis integrantes: dos desarrolladores encargados de la interfaz de usuario (*frontend*), tres enfocados en la lógica de negocio y bases de datos (*backend*), y un perfil dedicado a la infraestructura, operaciones y cultura DevOps (rol que asumí yo). Dada esta división de tareas, resultaba necesario establecer una estructura organizativa y técnica que permitiera el avance en paralelo de los distintos subequipos sin generar cuellos de botella ni dependencias bloqueantes. Por consiguiente, la adopción de la arquitectura que se describe a continuación no responde únicamente a criterios de rendimiento o escalabilidad tecnológica, sino a la necesidad crítica de coordinar eficientemente el trabajo simultáneo, aislar los entornos de desarrollo para evitar conflictos y garantizar una integración de código fluida y estandarizada.

La estructura base del repositorio quedaría tal se muestra:



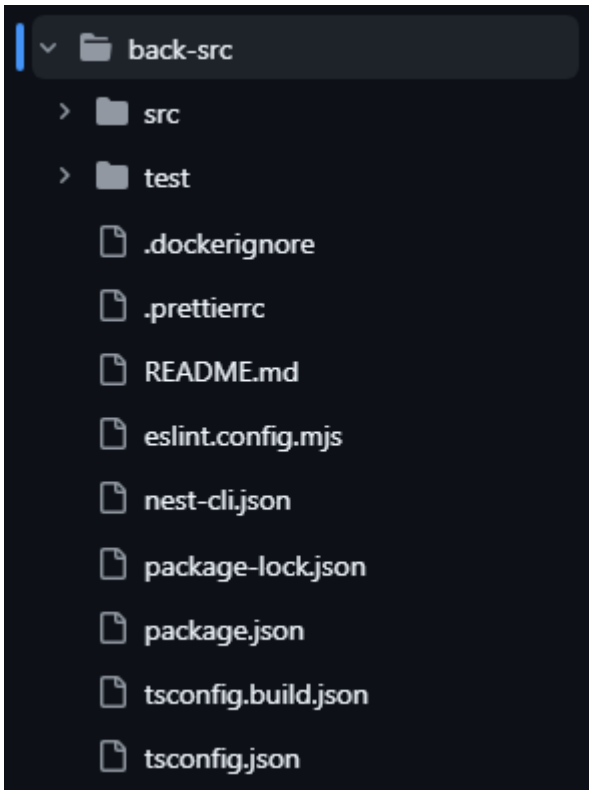
2.1. Adopción de la Arquitectura Monorepo y Separación de Contextos

Para la materialización de este escenario descrito, se ha optado por una arquitectura basada en microservicios y un enfoque de repositorio unificado o *Monorepo*. Esta decisión estratégica permite mantener todo el código fuente de la organización en un único lugar, facilitando la visibilidad global del proyecto y la estandarización de las herramientas de integración continua (CI/CD), que se abordarán en capítulos posteriores; a la vez que como hemos mencionado antes, permite el avance en paralelo de los equipos.

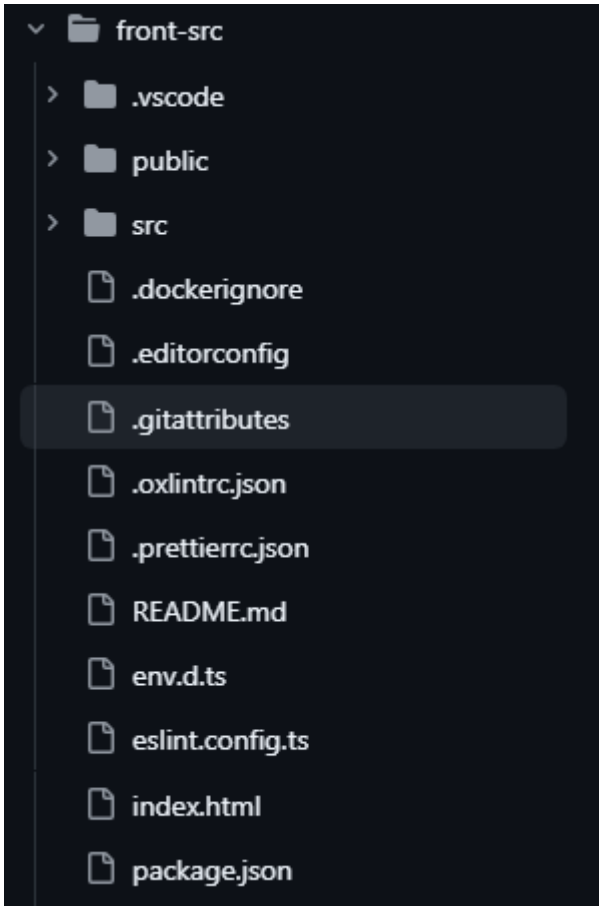
Sin embargo, para evitar el acoplamiento y garantizar la modularidad propia de la filosofía *Cloud Native*, se estableció una estricta separación de contextos desde la raíz del proyecto. La estructura se dividió en dos directorios principales y totalmente aislados:

- `front-src/`: Contiene el código del cliente (Frontend), desarrollado bajo un ecosistema basado en Vue, Vite y herramientas modernas de análisis de código estático (linters).
- `back-src/`: Aloja la lógica de negocio y la API (Backend), construida sobre el *framework* NestJS, gestionando conexiones a bases de datos y la validación de la lógica empresarial.

Contenedor del backend



Contenedor del frontend



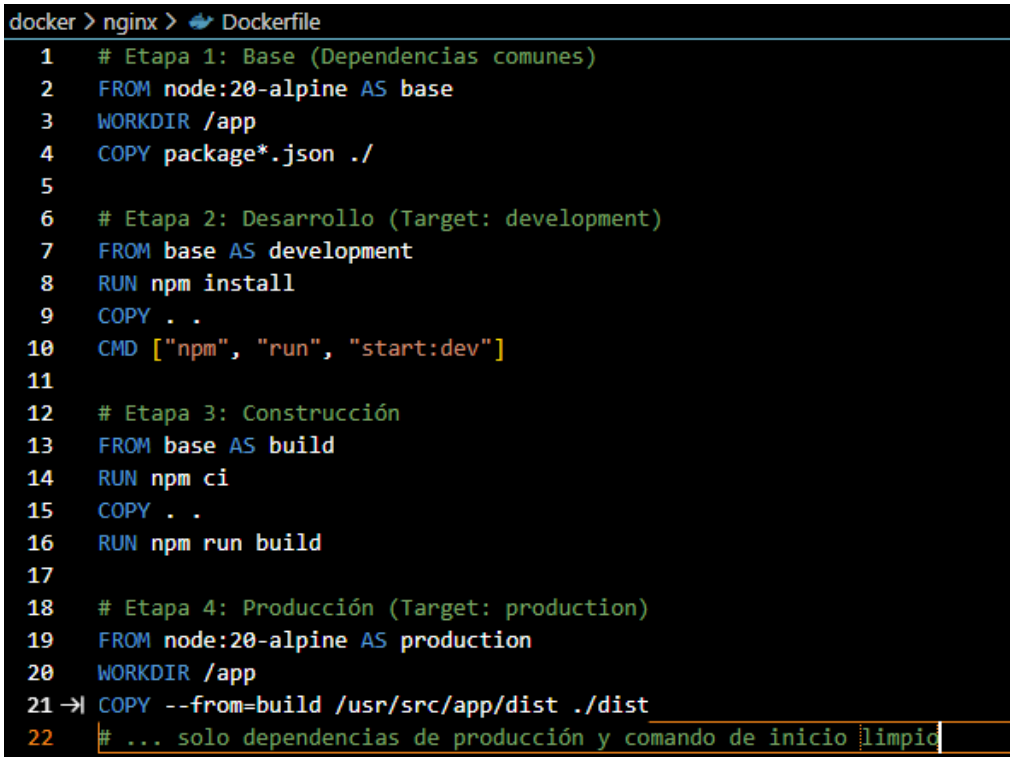
Un hito crítico en el diseño de esta arquitectura fue la decisión de aislar las dependencias. Al inicio del desarrollo, existía el riesgo de generar conflictos si ambos entornos compartían un único archivo de gestión de paquetes (`package.json`) en la raíz del proyecto. Para solucionarlo, se ha eliminado cualquier gestor de dependencias global. Actualmente, el frontend y el backend operan como entidades completamente autónomas, cada una con su propio `package.json` y `package-lock.json`. Esto garantiza que una actualización en las librerías de la base de datos del backend jamás interfiera con las dependencias de la interfaz de usuario del frontend, reduciendo drásticamente los conflictos en los *merges* entre los distintos equipos de desarrollo.

2.2. Diseño del Entorno de Desarrollo completo con Docker

Para trasladar el concepto de contenedorización al flujo de trabajo diario, se diseñó una infraestructura local avanzada utilizando **Docker** y **Docker Compose**. El objetivo no era únicamente levantar los servicios, sino crear una arquitectura que sirviera tanto para el desarrollo ágil como para el futuro paso a producción, evitando la duplicidad de código en las configuraciones.

2.2.1. Optimización de Imágenes con Multi-stage Builds

En lugar de utilizar archivos `Dockerfile` simples, se implementó el patrón de diseño **Multi-stage Build** (construcción en múltiples etapas). Esta técnica avanzada permite utilizar múltiples instrucciones `FROM` dentro de un mismo `Dockerfile`, separando lógicamente las fases de construcción, desarrollo y producción:



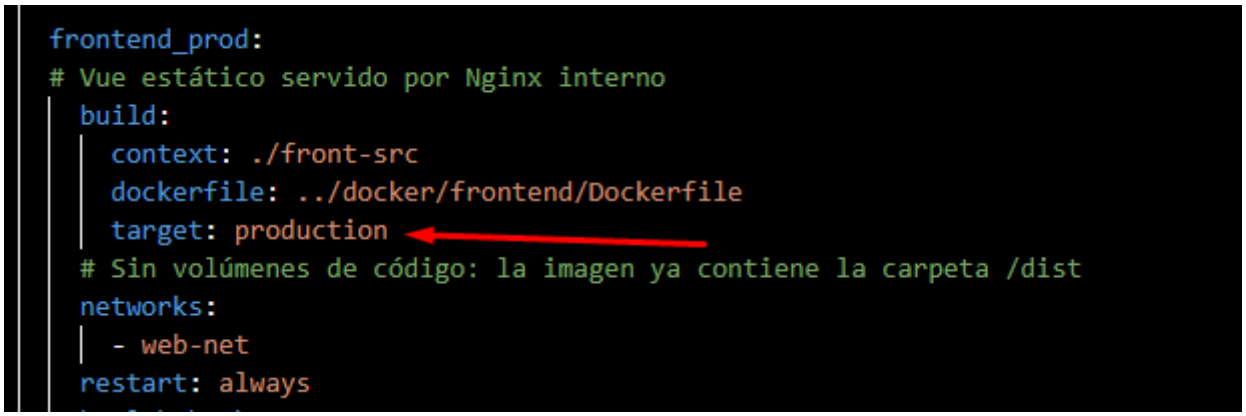
Esta decisión arquitectónica resuelve un problema fundamental: un entorno de desarrollo necesita herramientas de recarga en caliente (*hot-reload*), linters y dependencias de prueba, lo que genera imágenes pesadas; sin embargo, un entorno de producción debe ser lo más ligero y seguro posible, conteniendo exclusivamente el código compilado.

2.2.2. Orquestación Modular mediante múltiples Docker Compose

Para gestionar esta complejidad, se descartó el enfoque tradicional de un único archivo monolítico `docker-compose.yml`. En su lugar, la infraestructura se dividió en **tres archivos de orquestación separados**, aplicando el principio de modularidad.

Esta división estratégica permite separar la infraestructura base de las configuraciones específicas de cada entorno (Desarrollo vs. Producción) mediante la sobreescritura de configuraciones (*overrides*). El uso más destacado de esta técnica en el proyecto es la directiva `target`.

Al tener un `Dockerfile` con múltiples etapas, los archivos de Compose se configuran para detener la construcción en la etapa deseada, como por ejemplo:



De esta forma, el mismo `Dockerfile` sirve tanto para el equipo de desarrollo (inyectando el código fuente en tiempo real mediante volúmenes y apuntando al `target: development`) como para el CI/CD, asegurando una paridad total entre entornos.

2.2.3. Developer Experience (DX) y Scripts de Automatización

La fragmentación de la infraestructura en múltiples archivos Compose y el uso de comandos complejos de Docker añade una fricción operativa significativa para los programadores. Exigir al equipo la ejecución manual de comandos extensos (ej. combinando múltiples archivos con banderas `-f`) aumenta el riesgo de errores humanos y ralentiza el desarrollo.

Para mitigar esto y mejorar la Experiencia del Desarrollador (DX), se diseñó e implementó un **script de automatización** (mediante un archivo ejecutable tipo `.bat`). Este script actúa como una capa de abstracción operativa.

Funciones críticas que antes requerían un profundo conocimiento de Docker, como:

- Levantar el entorno de desarrollo combinando los archivos Compose correctos.
- Reconstruir imágenes forzando la limpieza de caché.
- Sincronizar las dependencias nativas del contenedor Linux con el entorno de trabajo local de Windows (como la sincronización del `package-lock.json` mencionada anteriormente).

Fragmento del archivo make.bat mostrando el uso de los comandos y el levantamiento en cascada de los contenedores y servicios:

```
:: 2. Enrutador de comandos validos
IF "%1"=="h" GOTO help
IF "%1"=="help" GOTO help
IF "%1"=="up" GOTO up
IF "%1"=="down" GOTO down
IF "%1"=="up-prod" GOTO up-prod
IF "%1"=="down-prod" GOTO down-prod
IF "%1"=="sync-back" GOTO sync-back
IF "%1"=="sync-front" GOTO sync-front

:: 3. Seguro contra fall-thru
echo [ERROR] El comando "%1" no es valido.
echo.
GOTO help

:up
echo [1/3] Comprobando red externa...
docker network inspect %NETWORK_NAME% >nul 2>nul || docker network create %NETWORK_NAME%
echo [2/3] Levantando Base de Datos...
docker-compose -f docker-compose.bd.yml up -d
echo [3/3] Levantando Aplicacion (Desarrollo)...
docker-compose up -d --build
echo ========================================
echo  ENTORNO DE DESARROLLO LISTO
echo ========================================
GOTO end

:down
echo Apagando Aplicacion (Desarrollo)...
docker-compose down
echo Apagando Base de Datos...
docker-compose -f docker-compose.bd.yml down
GOTO end
```

Una peculiaridad de nuestro diseño es que el contenedor de la base de datos está completamente independiente del stack de la aplicación. Es una decisión que radica en la posibilidad de una escalabilidad futura: que sea agnóstica y esté separada de los otros servicios de la aplicación permite migrar dicha base de datos a la nube por ejemplo, y que funcione y se integre sin problemas solo cambiando un parámetro en las variables de entorno. Al estar separada se necesitaba crear una red externa de docker para conectar ambos contenedores, por eso en el archivo make.bat primero se levanta la red externa, luego el contenedor de la base de datos y por último el stack de la aplicación.

Esta automatización no solo estandariza las rutinas del equipo, sino que encapsula la complejidad técnica, permitiendo que los desarrolladores de *frontend* y *backend* se concentren exclusivamente en la lógica de negocio sin tener que ser expertos en la administración de contenedores.

2.3. Gestión de Configuración y Seguridad: El paradigma de las Variables de Entorno

En el diseño de arquitecturas *Cloud Native*, la separación estricta entre el código fuente y la configuración es un principio innegociable. Siguiendo la metodología estándar de la industria conocida como *The Twelve-Factor App* (específicamente su tercer factor: "Guardar la configuración en el entorno"), se determinó que ninguna credencial, token de acceso, puerto o ruta de base de datos debía ser escrita de forma rígida (*hardcoded*) en el código de la aplicación.

Para implementar este principio en el entorno de desarrollo local, se adoptó el uso de archivos de variables de entorno (`.env`). Este enfoque presenta tres ventajas fundamentales para la operativa del equipo:

2.3.1. Aislamiento e Inyección en Contenedores

En lugar de que el código de Node.js o Vue intente adivinar dónde está la base de datos, estas configuraciones se externalizan. El archivo `docker-compose.yml` actúa como intermediario, leyendo el archivo `.env` del directorio local del desarrollador e inyectando esas variables de forma segura dentro de los contenedores en el momento de su arranque mediante la directiva `env_file` o el bloque `environment`.

De esta manera, el contenedor del backend puede conectarse a la base de datos utilizando variables dinámicas (como `DB_HOST` o `DB_PASSWORD`), permitiendo que el mismo código fuente exacto

funcione en el ordenador de un desarrollador, en el servidor de pruebas o en producción, simplemente cambiando el archivo `.env` que lo acompaña.

2.3.2. Seguridad y Control de Versiones

Desde el punto de vista de la seguridad (DevSecOps), los archivos `.env` contienen información altamente sensible. Por ello, una de las primeras políticas aplicadas en el repositorio fue la inclusión estricta de cualquier archivo `*.env` dentro del `.gitignore` global del proyecto. Esto previene fugas de credenciales accidentales en el control de versiones (repositorio de GitHub).

Para mantener la sincronización del equipo sin comprometer la seguridad, se estableció el uso de archivos plantilla denominados `.env.example`. Estos archivos sí se suben al repositorio y contienen la estructura de las variables necesarias, pero con valores vacíos o ficticios. Cuando un nuevo desarrollador se une al proyecto, únicamente debe duplicar esta plantilla, renombrarla a `.env` y solicitar las contraseñas reales al administrador del sistema.

.env.example

```
# ========================================
# PLANTILLA DE ENTORNO
# Instrucciones para Dev: Copia este archivo como '.env' y ajusta los valores si es necesario.
# NUNCA subas el archivo '.env' real a GitHub.
# ========================================

# --- 1. ENTORNO GLOBAL ---
NODE_ENV=

# --- 2. CONFIGURACIÓN DEL CONTENEDOR POSTGRESQL ---
# Estas variables las lee la imagen oficial de Postgres para crear la BD inicial
POSTGRES_USER=
POSTGRES_PASSWORD=
POSTGRES_DB=

# --- 3. CONFIGURACIÓN DE CONEXIÓN PARA NESTJS (Backend) ---
# NestJS usará estas variables para conectarse a la base de datos
DB_TYPE=
# ¡IMPORTANTE! El host no es localhost, es el nombre del servicio en el docker-compose
DB_HOST=
DB_PORT=
DB_USERNAME=
DB_PASSWORD=
DB_DATABASE=

# --- 4. CONFIGURACIÓN PARA VUE / VITE (Frontend) ---
# En Vite, todas las variables que queramos usar en el Frontend
# DEBEN empezar obligatoriamente por "VITE_"
VITE_API_URL=

```

2.3.3. Preparación para la Orquestación

La adopción de este paradigma en la fase de desarrollo local no es casual. Al acostumbrar a la aplicación a leer su configuración puramente desde el entorno (a través de `process.env` en Node.js, por ejemplo), el código queda arquitectónicamente preparado para su futuro despliegue en Kubernetes.

Como se detallará en capítulos posteriores, cuando la aplicación abandone el entorno local de Docker Compose, Kubernetes asumirá el control y sustituirá estos archivos `.env` físicos por sus propios recursos nativos de inyección de configuración: los *ConfigMaps* (para variables de entorno comunes) y los *Secrets* (para contraseñas y datos encriptados).

2.4. Homogeneización de Entornos y Gestión de Binarios Nativos

Uno de los mayores desafíos técnicos resueltos durante la fase de desarrollo fue la gestión de dependencias multiplataforma y la sincronización del `package-lock.json`.

En un equipo de desarrollo diverso, los programadores utilizan diferentes sistemas operativos subyacentes (Windows, macOS). Sin embargo, los entornos de Producción y los servidores de Integración Continua (como GitHub Actions) operan bajo distribuciones de Linux (ej. Ubuntu).

Esta discrepancia generó problemas críticos con herramientas modernas de alto rendimiento, como el linter *Oxlint* utilizado en el frontend. Al estar programadas en lenguajes compilados (como Rust o C++), estas herramientas requieren binarios nativos específicos para cada sistema operativo. Si un desarrollador instalaba las dependencias desde su máquina Windows, el archivo `package-lock.json` registraba únicamente los binarios para Windows. Posteriormente, cuando el servidor de Integración Continua (Linux) intentaba construir el proyecto basándose en ese archivo estricto, el proceso fallaba al no encontrar el binario compatible con su núcleo.

La solución arquitectónica implementada consistió en adoptar una política estricta de **"Operaciones dentro del Contenedor"**. En lugar de ejecutar comandos como `npm install` en la máquina local del desarrollador, se estableció que toda gestión de dependencias debe realizarse ejecutando el comando *dentro* del contenedor correspondiente (por ejemplo: `docker compose exec frontend npm install`).

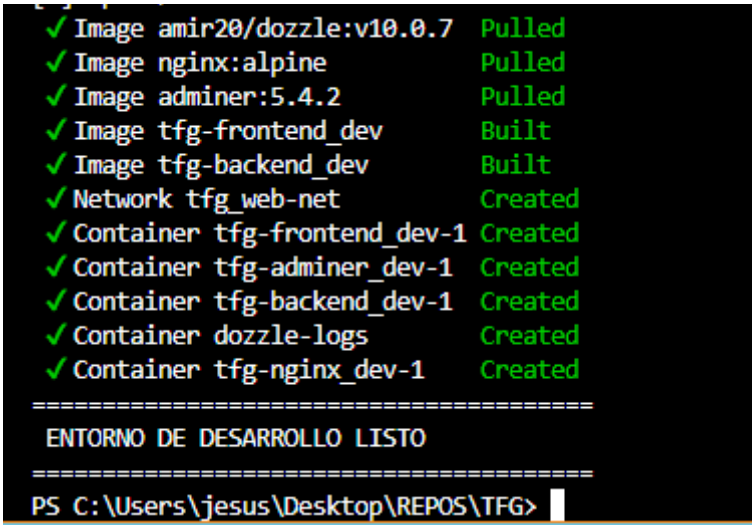
Al realizar esto, ocurren dos procesos vitales:

1. El motor de Node.js detecta que se encuentra dentro de un entorno Linux (el contenedor) y descarga los binarios nativos correctos para dicha arquitectura.
2. Gracias a los volúmenes compartidos, el archivo `package-lock.json` generado (que ahora contiene las referencias correctas para Linux) se sincroniza automáticamente con el disco duro del desarrollador, quedando listo para ser subido al repositorio de control de versiones.

Este procedimiento garantiza una paridad absoluta entre el entorno de desarrollo local, la fase de validación en la Integración Continua y el futuro despliegue en clústeres de Kubernetes, consolidando una base robusta para la automatización del ciclo de vida del software.

2.5. Resultado y arquitectura en funcionamiento

Para el día a día, el flujo de trabajo de los desarrolladores siempre va a comenzar de la misma forma: abren su IDE con la carpeta del repositorio, como aún estamos trabajando en local abrirán Docker-desktop y en la terminal de su IDE ejecutarán .\make.bat up:



Una vez el script de automatización termine su trabajo veremos esto, que significa que los contenedores de front, back y la base de datos se han levantado (así como otros servicios desplegados para ayudar a los desarrolladores como Adminer y Dozzle). Si ahora abriesen Docker desktop, veríamos cómo está funcionando nuestra infraestructura:



Name	Container ID	Image	Port(s)	CPU (%)	Last started
demo_database	-	-	-	0%	3 minutes ago
postgres-db	26456bf4fa89	postgres:16-alpine		0%	3 minutes ago
tfg	-	-	-	4.67%	2 minutes ago
dozzle-logs	bdbd4b83e1a7	amir20/dozzle:v10.0.7	8888:8080	0%	2 minutes ago
adminer_dev-1	2fb8303da4ae	adminer:5.4.2	8080:8080	0%	2 minutes ago
frontend_dev-1	904aef3cc44c	tfg-frontend_dev		0.4%	2 minutes ago
backend_dev-1	2cf21abf3ab8	tfg-backend_dev		4.27%	2 minutes ago
nginx_dev-1	034f5af33699	nginx:alpine	80:80	0%	2 minutes ago

Con esto acaba el despliegue de la infraestructura. De aquí en adelante los desarrolladores trabajarán a diario escribiendo código dentro de sus respectivas carpetas y subiendo el contenido a GitHub (o al repositorio que se use) y todo el trabajo colaborativo estará centralizado pero sin pisarse entre los equipos. Las mejoras en la infraestructura también se subirán y estarán disponibles cuando cada mañana hagan git pull de los cambios.

Capítulo 3: Integración Continua (CI) y Gestión de Artefactos

Una vez consolidado el entorno de desarrollo local mediante la orquestación con Docker Compose y la arquitectura Monorepo, el siguiente reto operativo consiste en definir cómo el código escrito por los desarrolladores se transforma de manera segura en un producto listo para ser desplegado. En un entorno colaborativo, depender de procesos manuales para la construcción de la aplicación genera cuellos de botella y un alto riesgo de error humano.

Para solventar esta problemática, se ha implementado un sistema de Integración Continua (CI) mediante GitHub Actions, diseñado para automatizar la validación, construcción y almacenamiento de las imágenes en un registro externo.

3.1. Transición del desarrollo local a la automatización (Pipelines)

El concepto de Integración Continua radica en la fusión temprana y frecuente del código en una rama principal compartida. Para materializar este proceso, se han diseñado flujos de trabajo automatizados (Pipelines). Estos pipelines actúan como una cadena de montaje virtual que se activa automáticamente (mediante *triggers*) cada vez que un desarrollador sube nuevo código al repositorio.

El objetivo de esta fase no es desplegar la aplicación directamente, sino garantizar que el código es funcional y generar un "artefacto" inmutable (la imagen de Docker) que posteriormente consumirá nuestra infraestructura en Proxmox.

3.2. Seguridad en la Nube: De archivos .env a GitHub Secrets

Uno de los principales retos al trasladar la construcción del proyecto a la nube es la gestión de credenciales. Como se expuso en el paradigma de diseño de la aplicación, existe una política estricta de incluir cualquier archivo *.env dentro del .gitignore global para prevenir fugas de información sensible.

Sin embargo, el servidor de Integración Continua necesita acceder a ciertas variables (como las URLs de las APIs o tokens de configuración) para poder ejecutar pruebas o, en el caso del Frontend

(Vite), para compilar la aplicación estática correctamente. Subir el archivo .env físico al repositorio rompería todas las políticas de seguridad.

La solución arquitectónica adoptada ha sido la utilización de **GitHub Secrets**. Esta herramienta permite almacenar variables de entorno de forma encriptada en los ajustes del repositorio. Durante la ejecución del pipeline, el servidor efímero de GitHub Actions extrae estos secretos y los inyecta en tiempo real como variables de entorno nativas, permitiendo compilar el código sin dejar rastro de las credenciales en el control de versiones.

3.3. Estrategia de Ramas, Control de Calidad y Prevención de Deuda Técnica

Para evitar cuellos de botella y asegurar la estabilidad de la aplicación, el pipeline se ha diseñado bajo una estrategia de flujos de trabajo condicionales, separando claramente la fase de Integración (control de calidad) de la fase de Entrega (publicación de artefactos).

Cuando un desarrollador trabaja en una nueva funcionalidad (en ramas tipo `feature/*` o `develop`), el objetivo principal es evitar la acumulación de deuda técnica. Al subir sus cambios o abrir una *Pull Request*, el pipeline ejecuta únicamente los *jobs* de validación:

1. **Análisis estático (Linting):** Validación estricta del código mediante herramientas como *Oxlint*.
2. **Pruebas Unitarias (Testing):** Ejecución de los test para garantizar la ausencia de regresiones.

Si cualquiera de estos procesos falla, la integración del código se bloquea, protegiendo la rama principal de código defectuoso. En esta fase no se construyen ni se suben imágenes a Docker Hub, ahorrando tiempo y recursos computacionales.

3.4. Optimización, Seguridad (DevSecOps) y Publicación

La construcción y publicación del artefacto se reserva exclusivamente para los eventos de fusión (*merge* o *push*) hacia la rama principal (`main`). Para esta fase crítica, el pipeline incorpora prácticas avanzadas de DevOps:

1. **Caché de Dependencias:** Se implementan mecanismos de caché tanto para los módulos de Node como para las capas de Docker. Esto evita descargar y compilar librerías inalteradas en cada ejecución, reduciendo drásticamente los tiempos del pipeline.
2. **Escaneo de Vulnerabilidades:** Siguiendo el paradigma DevSecOps establecido en la gestión de variables de entorno, antes de publicar la imagen final, esta es analizada por un escáner de seguridad (como *Trivy*). Si se detectan vulnerabilidades críticas (CVEs) en las dependencias del contenedor base, el proceso se aborta.
3. **Etiquetado Dinámico (Versioning):** Las imágenes no solo se etiquetan como `latest`, sino que reciben el identificador único del commit (SHA). Esto garantiza la trazabilidad absoluta: el equipo de operaciones siempre sabe exactamente qué versión del código se está ejecutando en producción.

A continuación, se muestra un diseño avanzado del pipeline reflejando esta separación de responsabilidades y condicionales:

```yaml
name: CI/CD Pipeline - Monorepo

# 1. DISPARADORES
on:
  push:
    branches: [ "main", "develop", "feature/*" ]
  pull_request:
    branches: [ "main" ]
  workflow_dispatch: # Permite ejecución manual

jobs:
  # ==========================================
  # FASE 1: INTEGRACIÓN Y CONTROL DE CALIDAD
  # ==========================================
  quality-checks:
    name: Lint & Test
    runs-on: ubuntu-latest
    steps:
      - name: Checkout del código
        uses: actions/checkout@v4

      - name: Setup Node.js (con caché)
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
          cache-dependency-path: '**/package-lock.json'

      - name: Validar Frontend (Oxlint & Tests)
        run: |
          cd front-src
          npm ci
          npm run lint
          npm run test

      - name: Validar Backend (Lint & Tests)
        run: |
          cd back-src
          npm ci
          npm run lint
          npm run test
```

```yaml
# FASE 2: CONSTRUCCIÓN, SEGURIDAD Y PUBLICACIÓN
# ==========================================
build-scan-push:
  name: Build & Push
  needs: quality-checks # Solo se ejecuta si la Fase 1 pasa
  if: github.ref == 'refs/heads/main' && github.event_name != 'pull_request' # Solo en merges/pushes a main
  runs-on: ubuntu-latest

  # Matriz para construir Front y Back en paralelo sin duplicar código
  strategy:
    matrix:
      include:
        - service: frontend
          context: ./front-src
          dockerfile: ./docker/frontend/Dockerfile
          image_name: jcmm13/tfg-frontend
        - service: backend
          context: ./back-src
          dockerfile: ./docker/backend/Dockerfile
          image_name: jcmm13/tfg-backend

  steps:
    - name: Checkout del código
      uses: actions/checkout@v4

    - name: Configurar Docker Buildx (Para Caché de capas)
      uses: docker/setup-buildx-action@v3

    - name: Autenticación segura en Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}

    - name: Etiquetado Dinámico (SHA y latest)
      id: meta
      uses: docker/metadata-action@v5
      with:
        images: ${{ matrix.image_name }}
        tags: |
          type=sha,format=short
          type=raw,value=latest

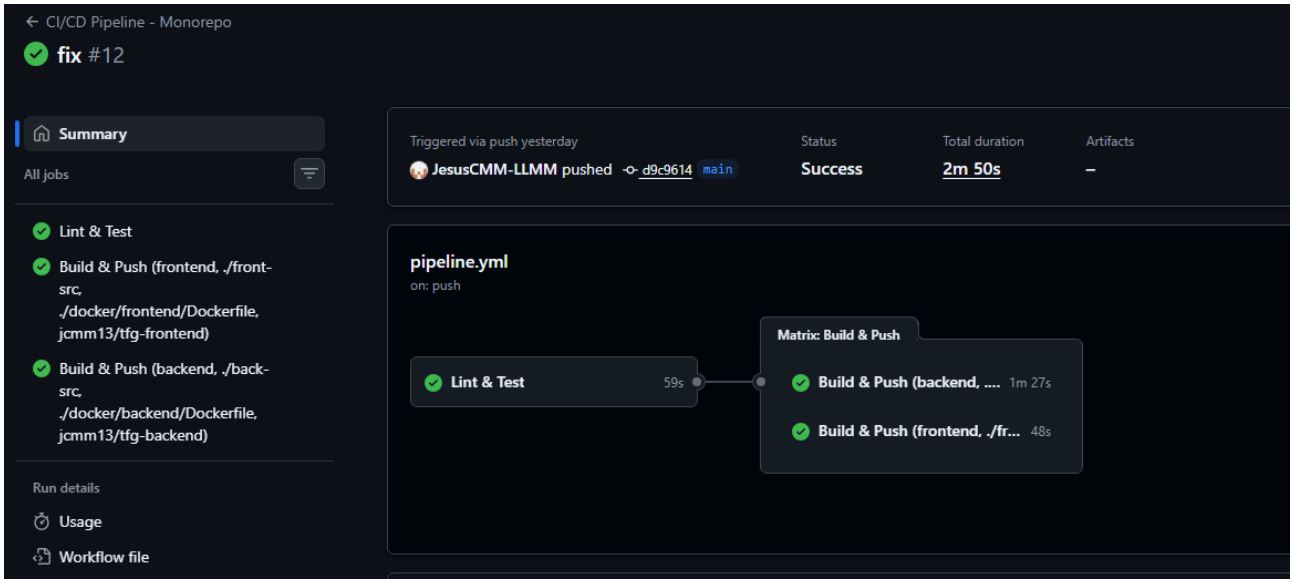
    - name: Construir Imagen Local (Target Production)
      uses: docker/build-push-action@v5
      with:
        context: ${{ matrix.context }}
        file: ${{ matrix.dockerfile }}
        target: production
        load: true # Mantiene la imagen en el runner para escanearla antes de subirla
        tags: ${{ steps.meta.outputs.tags }}
        cache-from: type=gha
        cache-to: type=gha,mode=max

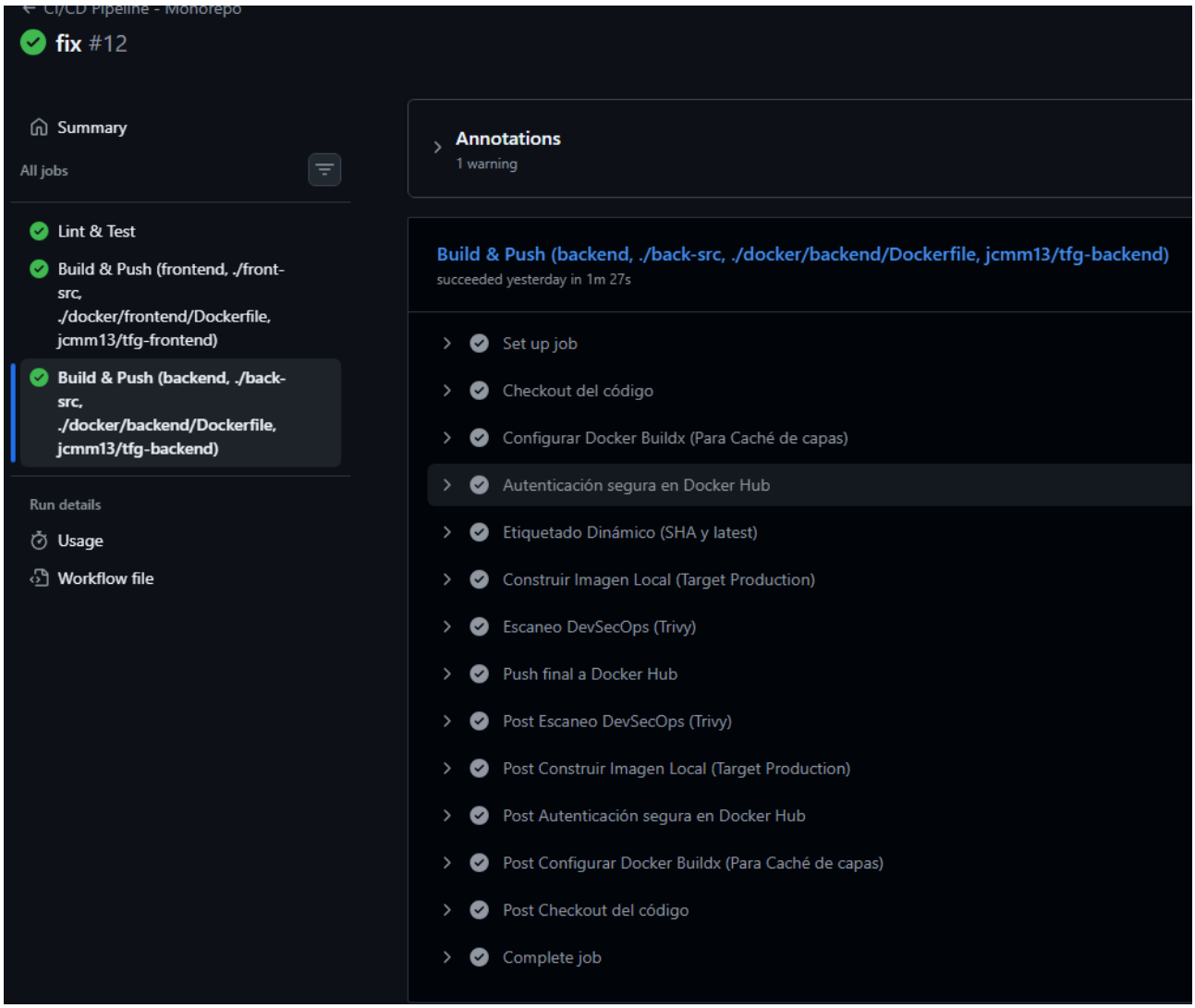
    - name: Escaneo DevSecOps (Trivy)
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: '${{ matrix.image_name }}:latest'
        format: 'table'
        exit-code: '1' # Falla el pipeline si encuentra vulnerabilidades críticas
        ignore-unfixed: true
        vuln-type: 'os,library'
        severity: 'CRITICAL,HIGH'

    - name: Push final a Docker Hub
      if: success() # Solo se ejecuta si Trivy no ha detectado fallos críticos
      run: docker push --all-tags ${{ matrix.image_name }}
```

Este archivo y todo el código relacionado puede verse directamente en el repositorio:

https://github.com/JesusCMM-LLMM/TFG





3.5. Justificación Técnica y Aportes de Valor del Pipeline

El diseño del flujo de trabajo automatizado detallado anteriormente no responde únicamente a la necesidad básica de compilar código, sino que integra prácticas avanzadas propias de la cultura DevOps y DevSecOps. A continuación, se exponen las decisiones arquitectónicas clave que dotan de robustez, eficiencia y seguridad a la infraestructura de Integración Continua:

- **Prevención Activa de la Deuda Técnica:** Al aislar el control de calidad en su propio entorno (`quality-checks`) y establecer una dependencia estricta para las fases posteriores (`needs: quality-checks`), se garantiza un ciclo de vida del software seguro. Si una actualización de código rompe una prueba unitaria o no supera el análisis estático, el proceso de despliegue se detiene de forma inmediata, protegiendo la integridad funcional del proyecto.
- **Aislamiento y Condicionales Estrictos:** El uso de directivas de validación de ramas (como `if: github.ref == 'refs/heads/main'`) actúa como una barrera física a nivel de infraestructura. Esto asegura categóricamente que los repositorios públicos de Docker Hub no puedan ser contaminados por imágenes inestables o versiones no definitivas provenientes de ramas de desarrollo secundarias.
- **Optimización de Recursos mediante Caché Avanzada:** La configuración del motor de construcción Docker Buildx para utilizar la caché nativa de la plataforma (`type=gha`) supone un salto cualitativo en eficiencia. Esta técnica almacena las capas inalteradas de los contenedores directamente en la memoria del repositorio, lo que reduce de manera drástica los tiempos de compilación en ejecuciones sucesivas y optimiza el consumo de recursos computacionales.
- **Implementación de Seguridad Ofensiva (DevSecOps):** La integración del escáner de vulnerabilidades Trivy, parametrizado intencionadamente con una política de tolerancia cero (`exit-code: '1'`), representa una postura proactiva. Si la herramienta detecta vulnerabilidades y exposiciones comunes (CVEs) de severidad alta o crítica en el sistema operativo base o en las librerías dependientes, el pipeline se aborta. Esto imposibilita la publicación de software vulnerable, garantizando que todo artefacto generado sea seguro por diseño.
- **Escalabilidad Paralela mediante Matrices de Construcción:** La adopción de la directiva `strategy: matrix` demuestra un enfoque avanzado para la automatización de arquitecturas complejas. Esta técnica permite al servidor procesar los microservicios de

frontend y *backend* de forma paralela y concurrente utilizando un único bloque lógico de instrucciones, cumpliendo rigurosamente con el principio de desarrollo DRY (*Don't Repeat Yourself*) y facilitando el mantenimiento a largo plazo de la infraestructura CI/CD.

3.6. Resolución de Conflictos Multiplataforma (Troubleshooting CI)

Durante el diseño del flujo de Integración Continua, se detectó una discrepancia crítica en la resolución de dependencias nativas entre el entorno de desarrollo y los servidores efímeros de GitHub Actions.

Al utilizar imágenes basadas en *Alpine Linux* para el entorno local y la construcción final, el gestor de paquetes de Node.js generaba un árbol de dependencias (`package-lock.json`) adaptado a la librería nativa de C `musl`. Sin embargo, los *runners* de integración continua de GitHub operan bajo entornos Ubuntu, los cuales requieren binarios compilados para `gnu`.

Esta fricción provocaba fallos de ejecución en herramientas de análisis estático programadas en lenguajes de bajo nivel (como Oxlint en Rust), manifestando el conocido **Bug #4828 de NPM**, el cual impide la resolución dinámica de dependencias opcionales multiplataforma.

Para solventar esta limitación técnica sin comprometer la inmutabilidad de la imagen de producción, se implementó un mecanismo de *bypass* en la fase de validación del pipeline (`quality-checks`). Se instruyó al servidor Ubuntu para purgar temporalmente el árbol de dependencias estricto y reevaluar el archivo `package.json` en tiempo de ejecución. Esto permite descargar los binarios `gnu` necesarios para superar las pruebas de calidad, garantizando un flujo de CI/CD ininterrumpido sin afectar a la fase final de construcción (Build), la cual sigue utilizando entornos Alpine.

Capítulo 4: Infraestructura como Código (IaC) y Despliegue en AWS EKS

Tras asegurar el ciclo de vida del software mediante la automatización de la Integración Continua (CI), el siguiente desafío consiste en desplegar la infraestructura física y lógica que alojará la aplicación. Siguiendo el paradigma *Cloud Native* detallado en el marco teórico, se ha optado por abandonar el aprovisionamiento manual de servidores en favor de la automatización total.

Para ello, se ha utilizado la nube pública de Amazon Web Services (AWS) y su orquestador gestionado **Amazon EKS (Elastic Kubernetes Service)**. Sin embargo, para mantener un control estricto sobre los recursos y aplicar una política de *FinOps* (reducción de costes operativos apagando la infraestructura cuando no se utiliza), todo el proceso se ha programado utilizando **Terraform**. Esta herramienta de Infraestructura como Código (IaC) permite crear, modificar y destruir el entorno de producción completo mediante comandos predecibles y versionados.

A continuación, se documenta de forma exhaustiva el registro de operaciones y el código necesario para levantar el clúster de Kubernetes y aplicar los manifiestos de la aplicación.

4.1. Preparación del Entorno de Aprovisionamiento

Antes de ejecutar cualquier código, es necesario establecer un canal de comunicación seguro y autenticado entre nuestra máquina local (o servidor de salto) y la API de AWS. Para este proyecto, las dependencias fundamentales instaladas son:

1. **AWS CLI:** La interfaz de línea de comandos oficial de Amazon. Se ha configurado ejecutando `aws configure` e inyectando las credenciales programáticas (*Access Key ID* y *Secret Access Key*) de una cuenta con permisos de administrador.
2. **Terraform (CLI):** El motor principal de HashiCorp encargado de interpretar los archivos `.tf` y planificar los cambios en la infraestructura.
3. **Kubectl:** La herramienta estándar de la industria para interactuar con el clúster de Kubernetes una vez que Terraform ha terminado de construirlo subyacentemente.

4.2. Aprovisionamiento detallado de la infraestructura con Terraform

El archivo `main.tf` constituye el punto de entrada de nuestra infraestructura y es el primer bloque de código que Terraform interpreta. Su función primordial es establecer los cimientos de la comunicación entre la herramienta de IaC y la API de Amazon Web Services, asegurando que el entorno sea predecible, seguro y versionado.

terraform-eks > main.tf

```
1   # main.tf - Configuración inicial y Proveedor
2   terraform {
3     required_providers {
4       aws = {
5         source  = "hashicorp/aws"
6         version = "~> 5.0"
7       }
8     }
9   }
10
11  provider "aws" {
12    region = "eu-west-1"
13    default_tags {
14      tags = {
15        Proyecto    = "TFG-ASIR"
16        Entorno     = "Produccion"
17        Gestionado = "Terraform"
18      }
19    }
20  }
```

Este bloque de configuración inicial se desglosa en dos secciones críticas: la declaración de requisitos de Terraform y la configuración específica del proveedor.

4.2.1. Bloque de configuración de Terraform (`terraform {}`)

Este segmento define las dependencias de software necesarias para que el motor de Terraform pueda interactuar con la nube de Amazon.

- **`required_providers`**: Dado que Terraform es una herramienta agnóstica a la nube, utiliza "plugins" o proveedores para comunicarse con diferentes plataformas. En este bloque, se especifica el uso del proveedor oficial de AWS desarrollado por HashiCorp.
- **Control de versiones (`version = "~> 5.0"`)**: Esta es una de las decisiones técnicas más importantes del proyecto. Al utilizar el operador de versión aproximada (`~>`), estamos garantizando que el proyecto utilice la versión 5.x del proveedor, permitiendo actualizaciones de parches de seguridad pero bloqueando cambios mayores (*breaking changes*) que podrían desestabilizar la infraestructura. Esto asegura la **reproducibilidad** del entorno a largo plazo.

4.2.2. Bloque del Proveedor de AWS (`provider "aws" {}`)

Una vez definida la dependencia, el bloque `provider` establece el contexto operativo y de autenticación en el que se desplegarán los recursos.

- **Definición de la Región (`region = "eu-west-1"`)**: Se ha seleccionado la región de Irlanda (`eu-west-1`) basándose en criterios de **latencia y disponibilidad de servicios**. Esta región es una de las más maduras de AWS en Europa y ofrece un equilibrio óptimo entre coste y rendimiento para los servicios de EKS y balanceadores de carga requeridos por el proyecto.
- **Etiquetado Global (`default_tags`)**: En lugar de etiquetar cada recurso (VPC, subredes, instancias) de forma individual y manual, se ha implementado una política de etiquetas por defecto.
    - Esta técnica es una piedra angular en las metodologías **FinOps** (Gestión Financiera de la Nube). Al inyectar etiquetas como `Proyecto`, `Entorno` y `Gestionado` de forma automática en cada recurso creado, se facilita enormemente la auditoría de costes y la identificación de recursos "huérfanos", permitiendo una administración mucho más eficiente y profesional del presupuesto del proyecto.

4.2.3. Proceso de Inicialización (`terraform init`)

Con este bloque definido, la ejecución del comando `terraform init` realiza tres tareas vitales:

1. Analiza el código del `main.tf`.
2. Descarga los binarios del proveedor de AWS en un directorio oculto (`.terraform`).
3. Crea el archivo de bloqueo de dependencias (`.terraform.lock.hcl`), el cual garantiza que cualquier otro miembro del equipo o servidor de despliegue utilice exactamente las mismas versiones de los plugins, eliminando el riesgo de inconsistencias entre entornos.

4.3. Arquitectura de Red: Despliegue de la VPC

La red es el componente más crítico en un despliegue de Kubernetes, ya que de ella depende tanto la comunicación interna entre microservicios como la exposición segura de la aplicación al exterior. En este apartado se detalla la configuración programada en el archivo `vpc.tf`, donde se ha definido una **Virtual Private Cloud (VPC)** diseñada específicamente para cumplir con los requisitos de alta disponibilidad y seguridad de Amazon EKS.

4.3.1. Segmentación de Red y Alta Disponibilidad

La VPC se ha configurado con un espacio de direccionamiento privado `10.0.0.0/16`. Para garantizar la resiliencia del sistema ante posibles fallos en un centro de datos de Amazon, se ha implementado una estrategia multizona:

- **Zonas de Disponibilidad (AZs)**: Mediante el recurso `data "aws_availability_zones"`, Terraform consulta dinámicamente las zonas activas en la región de Irlanda y selecciona automáticamente las dos primeras. Esto asegura que los recursos se distribuyan en infraestructuras físicas independientes.
- **Subredes Públicas (`10.0.101.0/24`...)**: Reservadas exclusivamente para los balanceadores de carga de entrada (*Ingress*). Son las únicas con acceso directo desde internet.

- **Subredes Privadas (`10.0.1.0/24`...)**: Aquí es donde residen los nodos de cómputo y las bases de datos. Al no tener una ruta directa desde el exterior, se reduce drásticamente la superficie de ataque, protegiendo la lógica de negocio.

4.3.2. Gestión de Salida a Internet y NAT Gateway

Aunque los nodos están en subredes privadas, necesitan acceder a internet para descargar actualizaciones de seguridad o imágenes de contenedores desde Docker Hub.

Para resolver este desafío, se ha implementado un **NAT Gateway** (Network Address Translation). Este componente actúa como un intermediario: permite que los recursos internos inicien conexiones hacia el exterior, pero impide que cualquier entidad externa inicie una conexión directa hacia ellos. Bajo una perspectiva de **FinOps**, se ha habilitado la opción `single_nat_gateway`, lo que permite compartir una única pasarela para todas las zonas de disponibilidad, optimizando los costes mensuales de infraestructura.

4.3.3. Etiquetas de Descubrimiento de Kubernetes

Un aspecto técnico avanzado de este despliegue es el uso de etiquetas (*Tags*) específicas en las subredes. Sin estas etiquetas, el controlador de Kubernetes en la nube no sabría dónde alojar los recursos de red:

- `kubernetes.io/role/elb = 1`: Identifica las subredes públicas donde AWS debe crear balanceadores de carga orientados a internet (Internet-facing).
- `kubernetes.io/role/internal-elb = 1`: Indica al orquestador dónde ubicar balanceadores internos si fuera necesario comunicar servicios privados entre sí.

Esta configuración permite que el despliegue de los manifiestos de Kubernetes (que veremos en el apartado 4.5) sea totalmente automático, ya que el clúster es capaz de "auto-descubrir" la infraestructura de red en la que se encuentra.

4.4. Orquestación y Cómputo: Despliegue del Clúster Amazon EKS

Con la arquitectura de red subyacente (VPC) completamente operativa, el siguiente paso crítico es el aprovisionamiento del orquestador de contenedores y la flota de máquinas virtuales que ejecutarán la aplicación. Toda esta lógica se ha encapsulado en el archivo `eks.tf`, utilizando el módulo oficial `terraform-aws-modules/eks/aws` para garantizar el cumplimiento de las mejores prácticas dictadas por Amazon.

terraform-eks > eks.tf

```
# eks.tf - Definición del Clúster de Kubernetes

module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name    = "tfg-cluster"
  cluster_version = "1.30" # Versión estable de Kubernetes (sin ser la más actual)

  # Conectamos el clúster a la VPC que creamos en vpc.tf
  vpc_id                   = module.vpc.vpc_id
  subnet_ids               = module.vpc.private_subnets
  control_plane_subnet_ids = module.vpc.private_subnets

  # Habilitado acceso público a la API para controlarlo desde la terminal local
  cluster_endpoint_public_access  = true

  # Permisos de administrador al usuario que está creando el clúster
  enable_cluster_creator_admin_permissions = true

  # Definición de las máquinas virtuales (Worker Nodes)
  eks_managed_node_groups = {
    nodos_tfg = {
      min_size     = 1
      max_size     = 5
      desired_size = 3

      instance_types = ["t3.small"]
      ami_type       = "AL2_x86_64" # <-- Especificamos explícitamente el sistema operativo base
    }
  }
}
```

4.4.1. El Plano de Control y la Gestión del Ciclo de Vida

El bloque principal del código define el *Control Plane* del clúster (el cerebro de Kubernetes). Se ha configurado bajo el nombre `tfg-cluster` y se le ha otorgado acceso público a su API (`cluster_endpoint_public_access = true`), lo que permite su administración remota y segura mediante la herramienta `kubectl` desde la terminal local del operador, validando las credenciales a través de AWS IAM.

Un aspecto fundamental en la administración de sistemas *Cloud Native* es la gestión del ciclo de vida del software. Durante la fase de desarrollo, se detectó que Amazon AWS impone políticas estrictas de obsolescencia programada (Deprecation) por motivos de seguridad. Al intentar aprovisionar el clúster con la versión `1.29`, la API de AWS devolvió una excepción (`InvalidParameterException`), indicando que las imágenes de sistema base (AMI) predeterminadas para esa versión ya no estaban soportadas para nuevos despliegues.

Como solución arquitectónica y para garantizar el soporte a largo plazo (LTS), se procedió a actualizar dinámicamente la versión del clúster a la **`1.30`** y se especificó explícitamente la familia de sistemas operativos para los nodos (`ami_type = "AL2_x86_64"`), asegurando la compatibilidad con Amazon Linux 2.

4.4.2. Integración Segura con la Topología de Red

Para garantizar que el clúster herede la seguridad perimetral definida en el apartado anterior, Terraform se encarga de inyectar dinámicamente los identificadores de la red. Mediante las variables `vpc_id` y `subnet_ids = module.vpc.private_subnets`, se fuerza a AWS a desplegar los nodos de trabajo (*Worker Nodes*) única y exclusivamente en las subredes privadas.

De este modo, los servidores que procesan la lógica de negocio y almacenan datos quedan totalmente aislados de la internet pública, siendo accesibles únicamente a través de los balanceadores de carga seguros de la capa de enrutamiento.

4.4.3. Cómputo: Managed Node Groups y Alta Disponibilidad

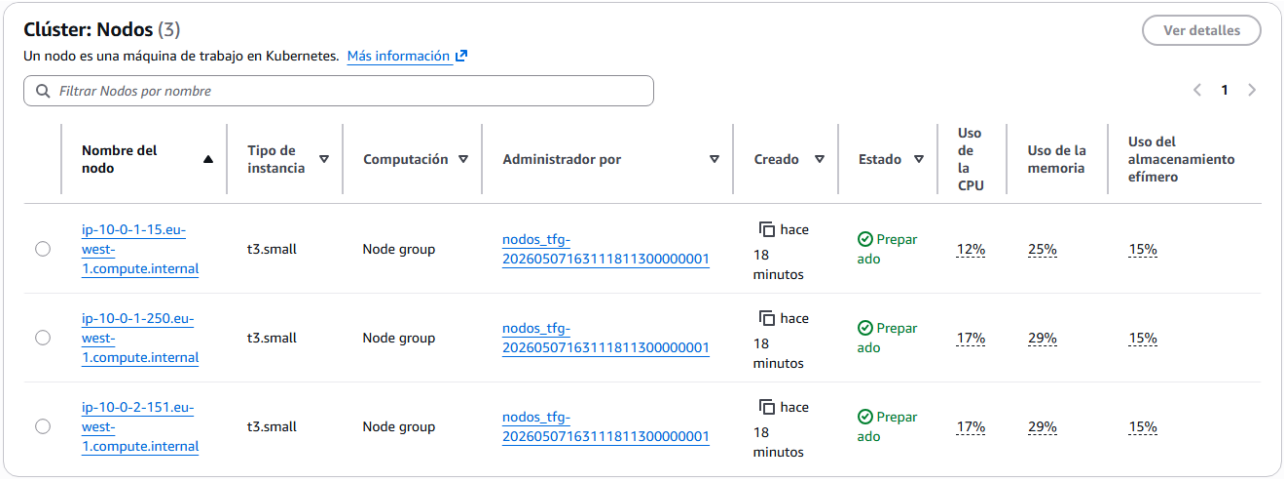
El músculo del clúster reside en su grupo de nodos gestionados (`eks_managed_node_groups`). Se ha definido una flota denominada `nodos_tfg` bajo los siguientes parámetros técnicos y de *FinOps*:

- **Tipo de Instancia:** Se han seleccionado máquinas virtuales de la familia `t3.small`. Estas instancias ofrecen un balance idóneo entre capacidad de cómputo (vCPUs y Memoria RAM) y eficiencia de costes, siendo perfectas para soportar los microservicios del proyecto (Frontend, Backend y Base de Datos) sin incurrir en sobredimensionamiento (*Overprovisioning*).

- **Escalabilidad y Tolerancia a Fallos:** Se ha configurado una capacidad deseada de **3 nodos** distribuidos entre las distintas Zonas de Disponibilidad. Esta configuración en clúster asegura la **Alta Disponibilidad (HA)**; si el hardware físico subyacente de uno de los nodos falla, Amazon EKS detectará la caída, aislará el nodo defectuoso y el *Scheduler* de Kubernetes redesplegará automáticamente los *Pods* huérfanos en los dos nodos restantes sin que el servicio experimente tiempos de inactividad perceptibles.

La aplicación de este código mediante el comando `terraform apply` supuso la orquestación simultánea de más de 50 recursos interdependientes en la nube, culminando con la creación exitosa del entorno de producción.

Una vez desplegado, podemos ver los nodos creados dentro del cluster en AWS:



Como desde nuestra consola local, con AWS CLI:

```
PS C:\Users\jesus> aws eks update-kubeconfig --region eu-west-1 --name tfg-cluster
Added new context arn:aws:eks:eu-west-1:951343842744:cluster/tfg-cluster to C:\Users\jesus\.kube\config
PS C:\Users\jesus> kubectl get nodes
NAME                                          STATUS   ROLES    AGE     VERSION
ip-10-0-1-15.eu-west-1.compute.internal       Ready    <none>   7m12s   v1.30.14-eks-ecaa3a6
ip-10-0-1-250.eu-west-1.compute.internal      Ready    <none>   7m12s   v1.30.14-eks-ecaa3a6
ip-10-0-2-151.eu-west-1.compute.internal      Ready    <none>   7m11s   v1.30.14-eks-ecaa3a6
PS C:\Users\jesus>
```

4.5. Orquestación de Contenedores: Despliegue Lógico de la Aplicación

Una vez aprovisionada la infraestructura física y el plano de control mediante Terraform, la última fase del despliegue consiste en inyectar la topología lógica de la aplicación en el clúster EKS. Para

ello, se ha utilizado la herramienta de administración estándar `kubectl`, autenticada previamente contra la API de AWS mediante el comando `aws eks update-kubeconfig`.

Fieles al principio de *Infraestructura como Código*, todos los recursos de Kubernetes se han declarado en formato YAML y organizado en un directorio dedicado (`k8s-manifests`). Para garantizar una resolución de dependencias correcta durante el arranque del sistema, los manifiestos se han estructurado y aplicado de forma secuencial:

1. **Aislamiento y Configuración (`00-namespace` y `01-config`):** En primer lugar, se crea un espacio de nombres lógico dedicado al proyecto, separando nuestros recursos de los del sistema (`kube-system`). A continuación, se inyectan los *ConfigMaps* y *Secrets* que contienen las variables de entorno, garantizando que ninguna contraseña viaje en texto plano en el código fuente.

```
PS C:\Users\jesus\Desktop\TFG> kubectl get nodes
NAME                                       STATUS   ROLES    AGE   VERSION
ip-10-0-1-15.eu-west-1.compute.internal    Ready    <none>   35m   v1.30.14-eks-ecaa3a6
ip-10-0-1-250.eu-west-1.compute.internal   Ready    <none>   35m   v1.30.14-eks-ecaa3a6
ip-10-0-2-151.eu-west-1.compute.internal   Ready    <none>   35m   v1.30.14-eks-ecaa3a6
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/00-namespace/
namespace/proyecto-tfg-2026 created
PS C:\Users\jesus\Desktop\TFG> kubectl get namespaces
NAME                STATUS   AGE
default             Active   54m
kube-node-lease     Active   54m
kube-public         Active   54m
kube-system         Active   54m
proyecto-tfg-2026   Active   9s
PS C:\Users\jesus\Desktop\TFG>
```

```
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/01-config/
configmap/backend-configmap created
secret/backend-secret created
PS C:\Users\jesus\Desktop\TFG> kubectl get configmaps,secrets -n proyecto-tfg-2026
NAME                           DATA   AGE
configmap/backend-configmap    5      32s
configmap/kube-root-ca.crt     1      88s

NAME                     TYPE     DATA   AGE
secret/backend-secret    Opaque   3      32s
PS C:\Users\jesus\Desktop\TFG>
```

2. **Capa de Persistencia (`02-database`):** Se despliega el motor de base de datos PostgreSQL. Este paso es crítico, ya que hace uso del controlador CSI de Amazon EBS para aprovisionar dinámicamente un disco duro virtual persistente en la nube, asegurando que los datos de la aplicación sobrevivan a la destrucción o reinicio de los *Pods*.

```
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/02-database/ebs-csi-sc.yaml
storageclass.storage.k8s.io/ebs-sc created
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/02-database/postgres-pvc.yaml
persistentvolumeclaim/postgres-pvc created
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/02-database/postgres-deployment.yaml
deployment.apps/proyecto-postgres created
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/02-database/postgres-service.yaml
service/proyecto-postgres created
PS C:\Users\jesus\Desktop\TFG> kubectl get pods,pvc -n proyecto-tfg-2026
NAME                                 READY   STATUS    RESTARTS   AGE
pod/proyecto-postgres-576dc86754-d5hl7   0/1     Running   0          29s

NAME                                   STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/postgres-pvc   Bound    pvc-36a60e40-4a7d-455b-b7a8-19f5d6a7cc20   2Gi        RWO            ebs-sc         <unset>                 38s
PS C:\Users\jesus\Desktop\TFG> kubectl get pods -n proyecto-tfg-2026 -w
NAME                                 READY   STATUS    RESTARTS   AGE
proyecto-postgres-576dc86754-d5hl7   1/1     Running   0          78s
```

3. **Lógica de Negocio e Interfaz (`03-backend` y `04-frontend`):** Se aplican los controladores de despliegue (*Deployments*). Es en este punto donde la orquestación se enlaza con la Integración Continua (CI): los manifiestos referencian directamente las imágenes Docker inmutables que nuestro pipeline de GitHub Actions compiló y subió a Docker Hub. Kubernetes descarga estas imágenes en los nodos `t3.small` y arranca los contenedores.

```
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/03-backend/backend-deployment.yaml
deployment.apps/proyecto-backend created
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/03-backend/backend-service.yaml
service/proyecto-backend created
PS C:\Users\jesus\Desktop\TFG> kubectl get pods -n proyecto-tfg-2026
NAME                                 READY   STATUS    RESTARTS   AGE
proyecto-backend-b7f7d9d8f-fglbt     1/1     Running   0          108s
proyecto-backend-b7f7d9d8f-gfb65     1/1     Running   0          108s
proyecto-postgres-576dc86754-d5hl7   1/1     Running   0          6m16s
PS C:\Users\jesus\Desktop\TFG>
```

```
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/04-frontend/
deployment.apps/proyecto-frontend created
service/proyecto-frontend created
PS C:\Users\jesus\Desktop\TFG> kubectl get pods -n proyecto-tfg-2026
NAME                                 READY   STATUS    RESTARTS   AGE
proyecto-backend-b7f7d9d8f-fglbt     1/1     Running   0          48m
proyecto-backend-b7f7d9d8f-gfb65     1/1     Running   0          48m
proyecto-frontend-65556696c5-22n6c   1/1     Running   0          11s
proyecto-frontend-65556696c5-4dth5   1/1     Running   0          11s
proyecto-postgres-576dc86754-d5hl7   1/1     Running   0          52m
PS C:\Users\jesus\Desktop\TFG>
```

4. **Exposición y Escalabilidad (`05-routing` y `06-hpa`):** Finalmente, se exponen los servicios internamente y se configuran las reglas de *Ingress* para enrutar el tráfico externo. Además, se aplican las reglas del *Horizontal Pod Autoscaler* (HPA), otorgando al sistema la capacidad de replicar los contenedores automáticamente en función del consumo de CPU, demostrando la verdadera elasticidad de una arquitectura *Cloud Native*.

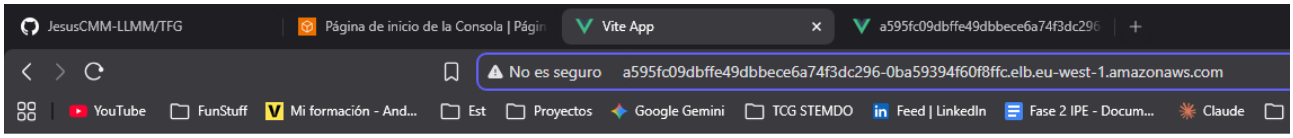
```
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/05-routing/ingress-rules.yaml
ingress.networking.k8s.io/proyecto-ingress created
PS C:\Users\jesus\Desktop\TFG> kubectl get ingress -n proyecto-tfg-2026
NAME              CLASS   HOSTS   ADDRESS   PORTS   AGE
proyecto-ingress   nginx   *                 80      13s
PS C:\Users\jesus\Desktop\TFG>

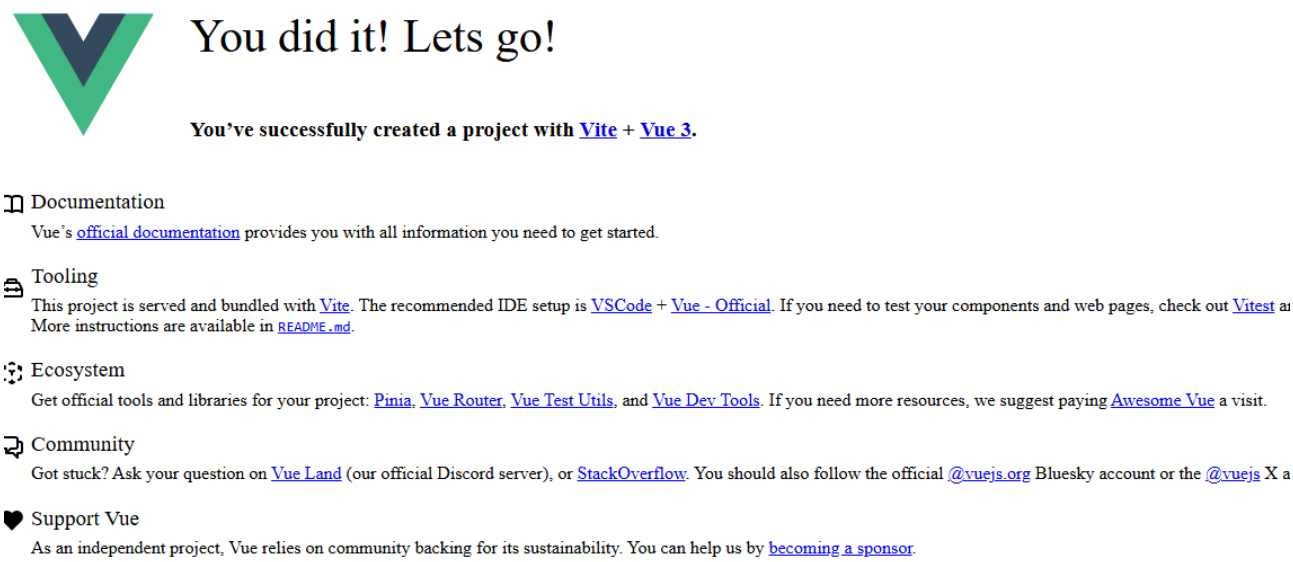
PS C:\Users\jesus\Desktop\TFG> kubectl apply -f k8s-manifests/06-hpa/hpa-rules.yaml
horizontalpodautoscaler.autoscaling/backend-hpa created
horizontalpodautoscaler.autoscaling/frontend-hpa created
PS C:\Users\jesus\Desktop\TFG> kubectl get hpa -n proyecto-tfg-2026
NAME           REFERENCE                     TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
backend-hpa    Deployment/proyecto-backend   cpu: 1%/60%   2         6         2          18s
frontend-hpa   Deployment/proyecto-frontend   cpu: 2%/60%   2         4         2          18s
PS C:\Users\jesus\Desktop\TFG>
```

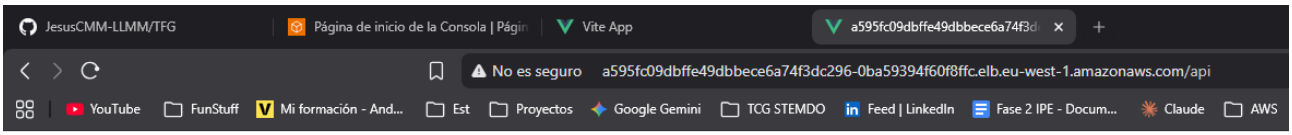
Ahora solo queda esperar a que el ingress nginx controller cree el balanceador de carga y nos de la dirección dns para acceder a nuestra aplicación.

```
PS C:\Users\jesus\Desktop\TFG> kubectl get ingress -n proyecto-tfg-2026
NAME              CLASS   HOSTS   ADDRESS                                                                   PORTS
   AGE
proyecto-ingress   nginx   *       a595fc09dbffe49dbbece6a74f3dc296-0ba59394f60f8ffc.elb.eu-west-1.amazonaws.com   80
   2m36s
PS C:\Users\jesus\Desktop\TFG>
```

Este despliegue secuencial culmina el ciclo de vida del software, transformando el código fuente alojado en un repositorio en una aplicación altamente disponible, pública y tolerante a fallos operando en la nube de Amazon.






JesusCMM-LLMM/TFG
Página de inicio de la Consola | Págir
Vite App
a595fc09dbffe49dbbece6a74f3d
No es seguro
a595fc09dbffe49dbbece6a74f3dc296-0ba59394f60f8ffc.elb.eu-west-1.amazonaws.com/api
YouTube
FunStuff
Mi formación - And...
Est
Proyectos
Google Gemini
TCG STEMDO
Feed | LinkedIn
Fase 2 IPE - Docum...
Claude
AWS

Hello World!

Capítulo 5: Gestión Visual del Clúster con Portainer

5.1. Introducción a Portainer

A pesar de que la administración de Kubernetes se realiza habitualmente mediante la interfaz de línea de comandos (CLI) con `kubectl`, en entornos de producción es altamente recomendable disponer de una interfaz gráfica (GUI) que permita una monitorización rápida y una gestión visual de los recursos.

Para este proyecto se ha seleccionado **Portainer**, una plataforma de gestión de contenedores ligera y potente que permite abstraer la complejidad de los archivos YAML y visualizar el estado del clúster en tiempo real.

5.2. Proceso de Instalación

La instalación de Portainer se ha realizado mediante **Helm**, el gestor de paquetes estándar de Kubernetes. Este método garantiza una instalación limpia y fácilmente actualizable.

5.2.1. Configuración del Repositorio

En primer lugar, se añadió el repositorio oficial de Portainer a nuestro entorno local y se procedió a actualizar los índices de Helm con los comandos:

```
helm repo add portainer https://portainer.github.io/k8s/
helm repo update
```

5.2.2. Despliegue en el clúster

Para el despliegue, se optó por crear un espacio de nombres (*namespace*) dedicado llamado `portainer`. Además, se configuró el servicio como tipo **LoadBalancer** para que AWS EKS aprovisionara automáticamente un balanceador de carga externo (NLB), permitiendo así el acceso desde el navegador:

```
helm install portainer portainer/portainer -n portainer --create-namespace --set service.type=LoadBalancer
```

5.2.3. Acceso y Seguridad Inicial

Una vez desplegado, AWS asignó un nombre DNS al servicio. Debido a que Portainer utiliza certificados auto-firmados por defecto, el acceso se realizó a través del puerto **9443 (HTTPS)**.

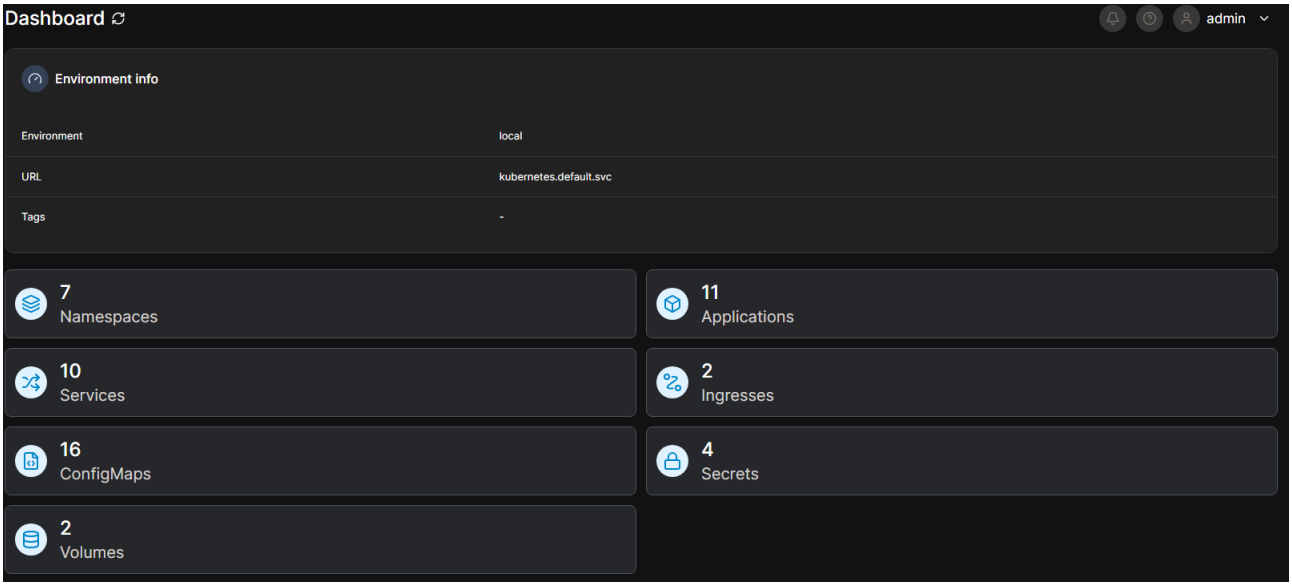
Nota técnica: Portainer incorpora una medida de seguridad que bloquea la instancia si no se configura la contraseña de administrador en los primeros 5 minutos tras el despliegue. En caso de bloqueo, es necesario reiniciar el despliegue mediante un `rollout restart`.

5.3. Monitorización y Administración del Proyecto

Tras completar la configuración inicial, Portainer permite una visibilidad completa de los recursos del TFG dividida en varias áreas clave:

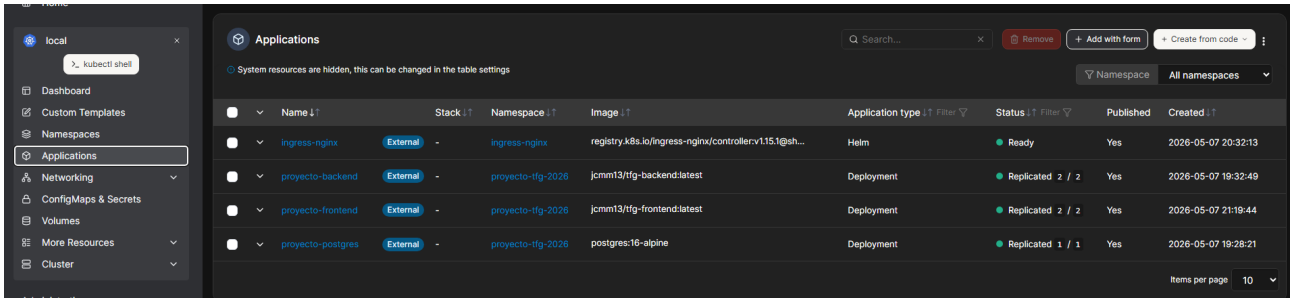
5.3.1. Dashboard General

El panel principal ofrece una visión holística del clúster. Aquí se pueden monitorizar de un vistazo el número de *Namespaces*, *Stacks* (aplicaciones), *Volumes* y *Secrets* activos. Es la primera línea de defensa para detectar anomalías globales.



5.3.2. Gestión de Aplicaciones (Namespaces)

Dentro del apartado de **Applications**, podemos filtrar por nuestro namespace específico (`proyecto-tfg-2026`). Portainer muestra visualmente el estado de salud de cada componente de nuestra arquitectura:

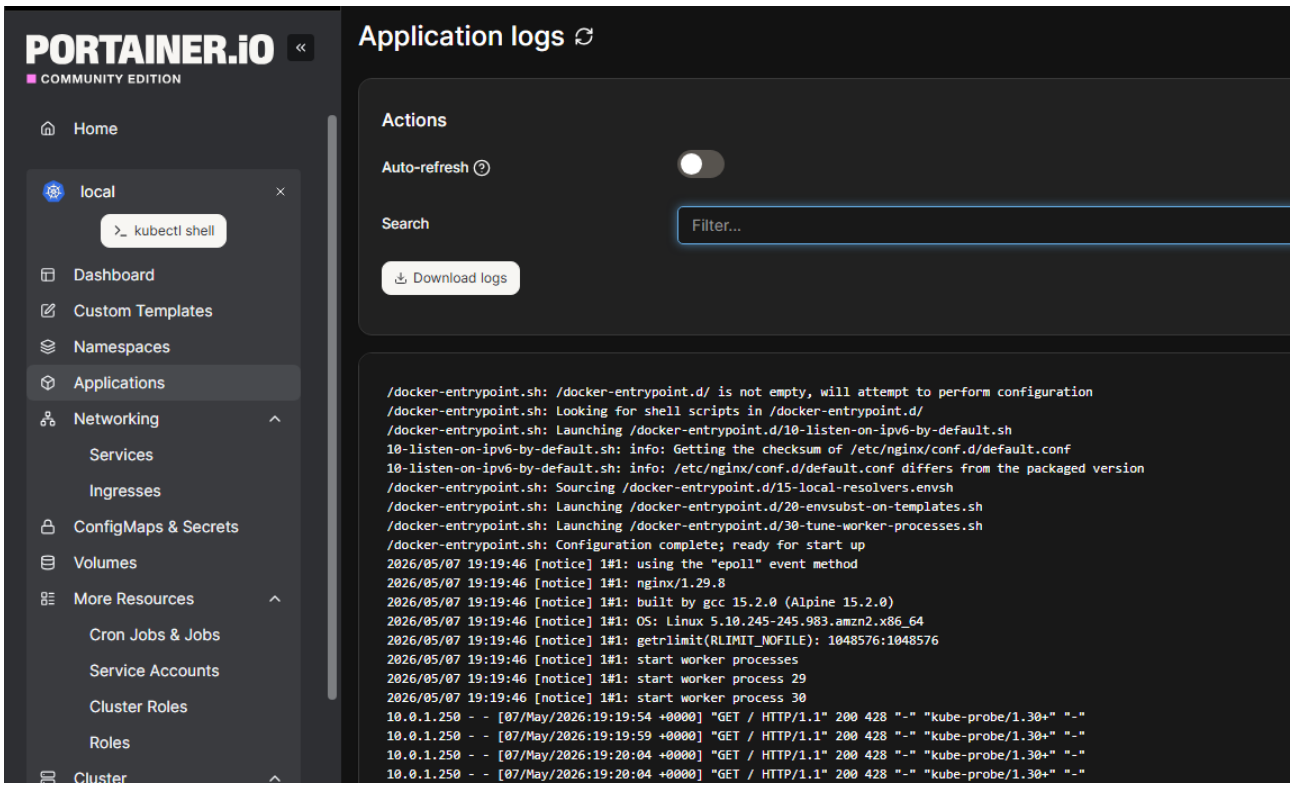


- **Frontend:** Despliegue de Nginx con Vue.js.
- **Backend:** API en NestJS.
- **Database:** Instancia de PostgreSQL.

Desde esta vista, el administrador puede escalar el número de réplicas, editar la configuración en vivo o reiniciar servicios con un solo clic.

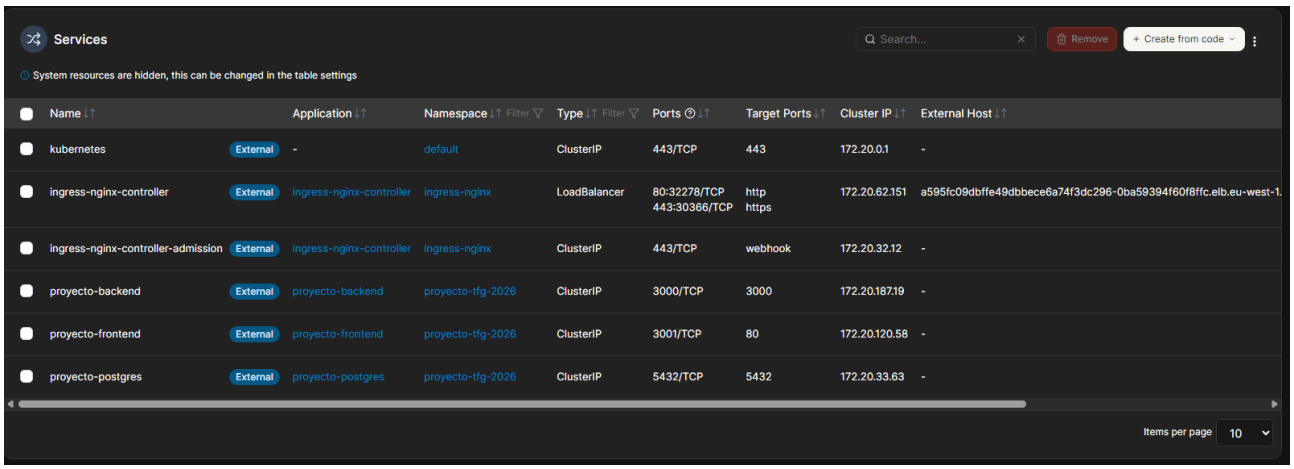
5.3.3. Inspección de Logs y Consola

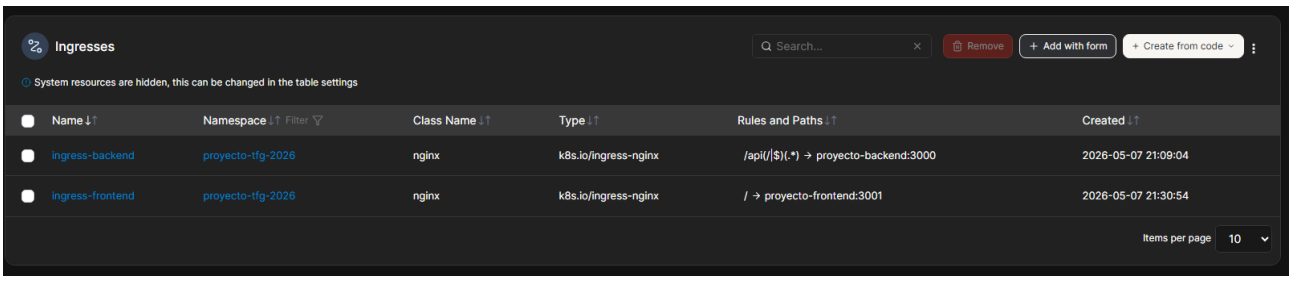
Una de las funcionalidades más potentes para un técnico ASIR es la capacidad de acceder a los **Logs** de los contenedores sin salir del navegador. Esto ha permitido, durante el desarrollo del proyecto, depurar los errores de conexión entre el Ingress y el Frontend de manera mucho más ágil que mediante la terminal. Además, Portainer permite abrir una **Consola Interactiva** dentro de cualquier Pod para realizar pruebas de conectividad de red interna.



5.3.4. Networking e Ingress

Portainer mapea visualmente nuestras reglas de **Ingress**. En este apartado se puede verificar que las rutas `/` y `/api` están correctamente dirigidas a los servicios correspondientes, facilitando la auditoría de la red externa del clúster.





Capítulo 6: Conclusiones y líneas de mejora futuras

6.1. Conclusiones técnicas

Tras la finalización del proyecto, se ha cumplido el objetivo principal: el despliegue de una infraestructura de microservicios robusta, escalable y administrable en la nube de Amazon Web Services (AWS). Las conclusiones técnicas más relevantes son:

- **Eficacia de la Infraestructura como Código (IaC):** El uso de Terraform ha permitido estandarizar el despliegue de recursos complejos (VPC, Subnets, EKS, Nodos), reduciendo errores humanos y garantizando que el entorno sea reproducible.
- **Gestión de Redes en Cloud:** Se ha demostrado que la configuración de la red en entornos EKS es uno de los mayores retos. La correcta gestión de los *Security Groups* y la comprensión del flujo de tráfico desde el *Load Balancer* hasta los *Pods* es crítica para la disponibilidad del servicio.
- **Abstracción y Operatividad:** La implementación de Portainer ha validado que es posible democratizar la administración de Kubernetes, permitiendo que perfiles menos técnicos puedan monitorizar y depurar servicios sin depender exclusivamente de la CLI.

6.2. Líneas de mejora futuras

Dada la naturaleza escalable del proyecto, se proponen las siguientes mejoras para una fase posterior:

1. **Implementación de CI/CD:** Automatizar el ciclo de vida del software mediante GitHub Actions y Terraform, para que cualquier cambio en el código del frontend o backend se despliegue automáticamente en el clúster apoyado en los manifiestos ya escritos.
2. **Seguridad Avanzada (SSL/TLS):** Integrar *Cert-Manager* con *Let's Encrypt* para cifrar todo el tráfico mediante certificados HTTPS válidos, eliminando los avisos de seguridad en el navegador.

3. **Monitorización con Prometheus y Grafana:** Aunque Portainer ofrece una gestión visual excelente, la implementación de un stack de monitorización con Prometheus permitiría establecer alertas automáticas (ej. avisar por Telegram si un nodo se queda sin RAM).
4. **Optimización de Costes:** Utilizar *AWS Spot Instances* para los nodos del clúster, lo que podría reducir la factura mensual de AWS en hasta un 70-90% para entornos de prueba.

6.4. Diagrama de arquitectura final

Para finalizar, se presenta el esquema lógico de la solución implementada. Este diagrama muestra el viaje de una petición desde el usuario hasta la base de datos, atravesando todas las capas de seguridad y orquestación configuradas.

